

## Appunti del 05-03-26 Mattina: Brutta Copia – PROGETTO BLOGGER

1. Bisogna imparare a gestire la complessità

2. Analizzare i requisiti

Bisogna capire chi e cosa.

Il programma funziona in questo modo: Vogliamo creare una piattaforma da offrire a chi vuole aprire un blog. Con AI integrata, per fare “una censura preventiva”, per moderare i contenuti del sito che ospitiamo.

Cominciamo a ragionare sui profili, sugli attori che useranno il nostro locale

### Visitatore

Guarda la HomePage e può cercare per blogger/blog/per argomento.

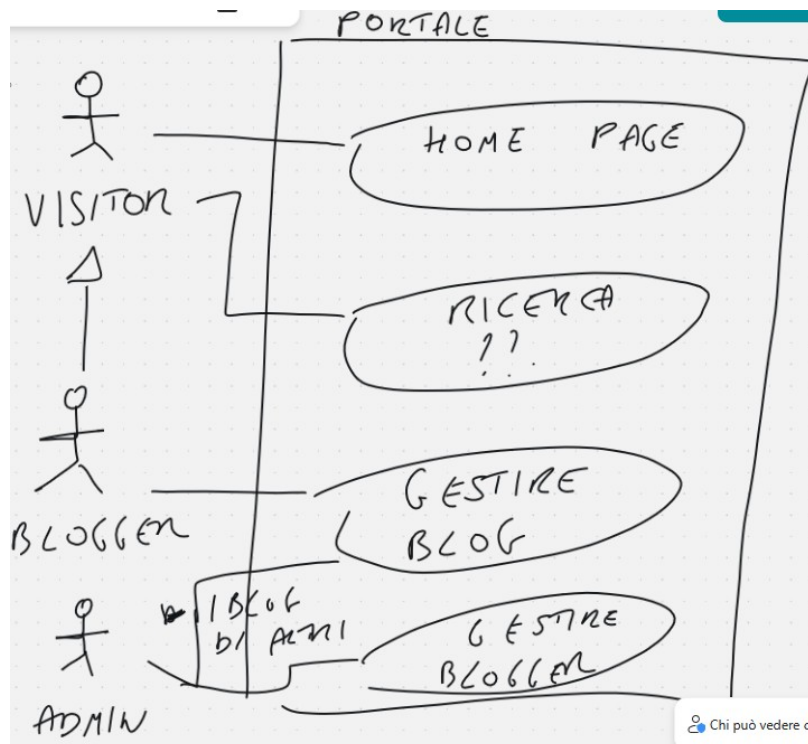
Il visitor, potrà vedere la home page, dove ci saranno gli ultimi articoli. Useremo l'AI per fare dei mockup / idee.

Il visitor può fare della ricerca (?? non abbiamo ancora deciso come ricercare).

Bisogna registrarsi come “autore” e scrivere qualcosa (come ad esempio un blog sul log di questo progetto).

Un visitor viene esteso da un blogger, che è allo stesso tempo un attore (uno che usa il sistema) ed un'entità. Un blogger può gestire un blog.

L'admin non estende blog, l'admin può gestire i blog di altri.



Non si può perdere tempo per cercare la soluzione migliore, perché quando l'avremo trovata sarà già obsoleta.

### DATA MODEL

E' fondamentale capire di cosa parliamo, per primo:

USER → con un campo categoria (blogger, admin), oltre agli altri campi da decidere

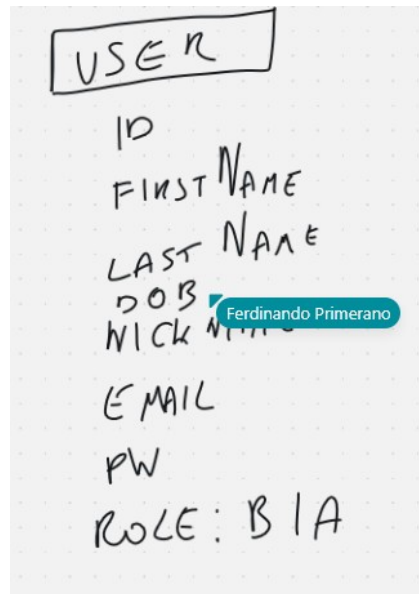
Responsabilità penale, cosa vuol dire. Gli utenti devono essere identificati, mi servono nome, cognome, data di nascita (possibilità la carta di identità), altra cosa importante, nickname, email e password.

Un blog è un insieme di post con tema comune. I blog hanno una relazione n a 1 con uno user.

Un blog ha n-BlogPost

con un blog post ci sono -n commenti.

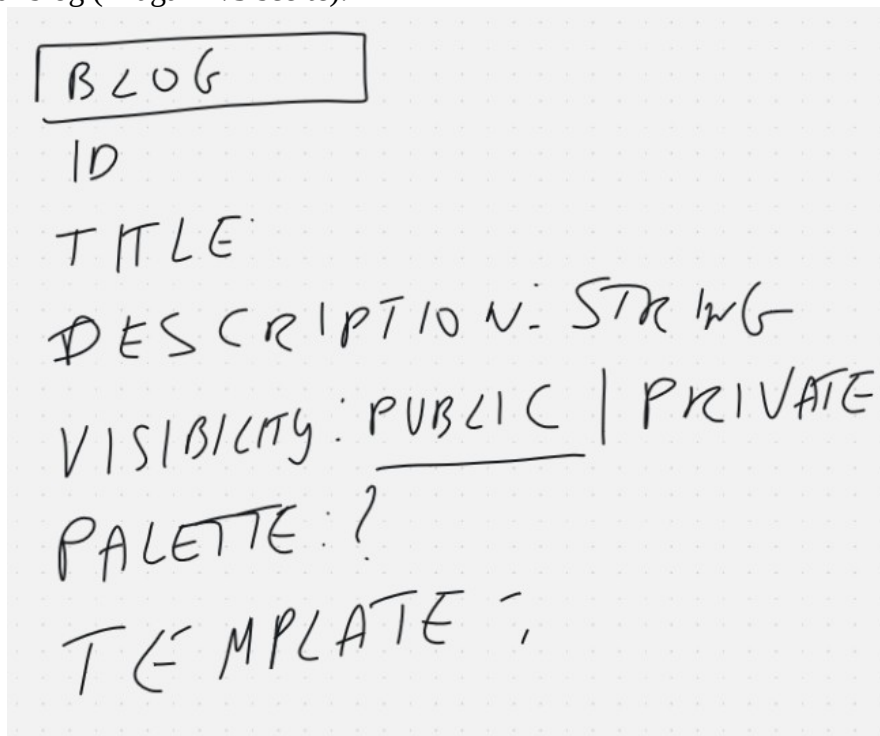
**La politica dei commenti**, L'AUTORE può decidere se accettare o meno commenti sul singolo post.



L'admin è obbligato a cambiare la password ogni due settimane (bisogna rispondere a domande come del tipo, dov'è il cambio password, o domande inerenti alla logica)

Il blogger può anche non cambiare la password.

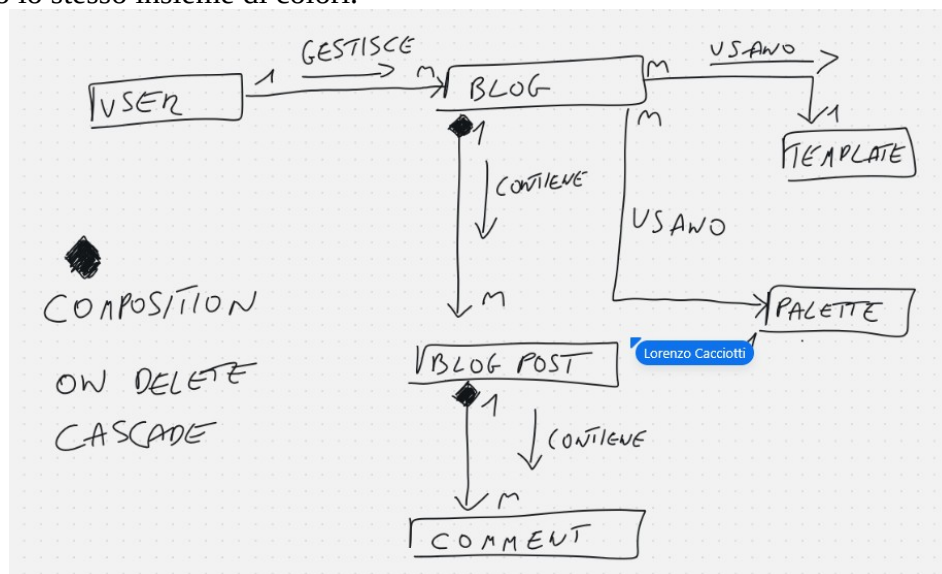
Che cos'è un **BLOG**, ogni blog non devono essere tutti uguali, devono poter cambiare colore o template, stile di blog (magari 2/3 scelte).



Un blog può avere una visibilità.

Nel backend io memorizzo le scelte dell'utente. . TAGS (argomenti)

Uno user gestisce N blog. Un blog usano 1 template.  
N blog usano lo stesso insieme di colori.



Composizione sensu strictu.

La **composizione** significa esattamente questo: **l'entità figlia non può esistere senza l'entità padre**. Se il padre viene eliminato, anche i figli vengono eliminati a cascata.

L'**integrità referenziale** è il vincolo del database che garantisce che le relazioni tra tabelle rimangano consistenti. **Blog Post** → **Blog** La chiave esterna `blog_id` in `BlogPost` deve sempre riferirsi a un `Blog` esistente. Non puoi inserire un `BlogPost` con un `blog_id` che non esiste nella tabella `Blog`.

**Comment** → **Blog Post** La chiave esterna `post_id` in `Comment` deve sempre riferirsi a un `BlogPost` esistente. Non puoi inserire un `Comment` con un `post_id` inesistente.

Operazioni critiche da gestire:

**INSERT** → prima crei il padre, poi il figlio

- Prima crei il `Blog`, poi il `BlogPost`, poi il `Comment`

**DELETE** → dato che è composizione, si usa **CASCADE**

- Elimini il `Blog` → si eliminano automaticamente tutti i suoi `BlogPost` e di conseguenza tutti i `Comment`
- Elimini il `BlogPost` → si eliminano automaticamente tutti i suoi `Comment`

**UPDATE** → se aggiorni la chiave primaria del padre (raro), anche la FK del figlio deve aggiornarsi (**ON UPDATE CASCADE**)

La composizione nel diagramma UML si mappa quindi direttamente su **ON DELETE CASCADE** nel database, proprio perché la figlia non ha senso di esistere senza la madre.

```

1 package com.generation.voices;
2
3 import java.time.LocalDate;
4
5 import lombok.Data
6     Source: lombok-1.18.42.jar
7 @Data
8 public class PortalUser {
9
10     private int id;
11     private String firstName;
12     private String lastName;
13     private LocalDate dob;
14     private String email;
15     private String password;
16     private Role role;
17
18 }

```

@data → to string, getter e setter



```

> main > java > com > generation > voices > Role.java > Role > ADMIN
1 package com.generation.voices;
2
3
4 public enum Role {
5     BLOGGER,
6     ADMIN
7 }

```

privato vuol dire su invito ( il lblog)

Gestione dei tempi: per domani alle ore 13 l'immagine di un blog funzionante  
NON AVREMO ANCORA IL SISTEMA DI LOGIN, non avremo ancora la maggior parte delle api

SCADENZA 06/03/26

una pagina web con BLOG DI PROVA, inserito a mano dall'admin, con dati di prova, ma che dia un'idea di come funzionerà il tutto  
solo lettura: API GET /voices/blogs/1

voices/blogs/"titolo del blog ipotizzando si chiami la strada\_e\_i\_canti"

```
// scadenza 6 marzo ore 13
// una pagina web con un BLOG DI PROVA
// inserito a mano dall'admin
// con dati di prova
// ma che dia un'idea di come funzionerà il tutto
// solo lettura: API GET /api/voices/blogs/1
// voices/blogs/la_strada_e_i_canti (percorso verso il componente BlogPage di Angular)
```

di conseguenza anche il titolo del blog deve essere univoco.

Scelgo un punto di arrivo, credo un punto di arrivo e ci arrivo, non tutto il suo sistema.

---

@Valid è un'annotazione Java (Jakarta Bean Validation) usata in Spring Boot per **validare automaticamente** i dati in ingresso — tipicamente il body di una richiesta POST/PUT.

@RequestBody dice a Spring di **leggere il corpo (body) della richiesta HTTP** e convertirlo automaticamente in un oggetto Java (deserializzazione JSON → oggetto).

ResponseBody<T> è un oggetto Spring che ti permette di **controllare completamente la risposta HTTP** — non solo il body, ma anche lo **status code** e gli **headers**.

// 200 OK

ResponseBody.ok(user)

// 201 Created

ResponseBody.status(201).body(savedUser)

ResponseBody.created(uri).body(savedUser)

// 204 No Content (es. dopo una DELETE)

ResponseBody.noContent().build()

// 400 Bad Request

ResponseBody.badRequest().body("Dati non validi")

// 404 Not Found

ResponseBody.notFound().build()

// 500 Internal Server Error

ResponseBody.internalServerError().body("Errore server")